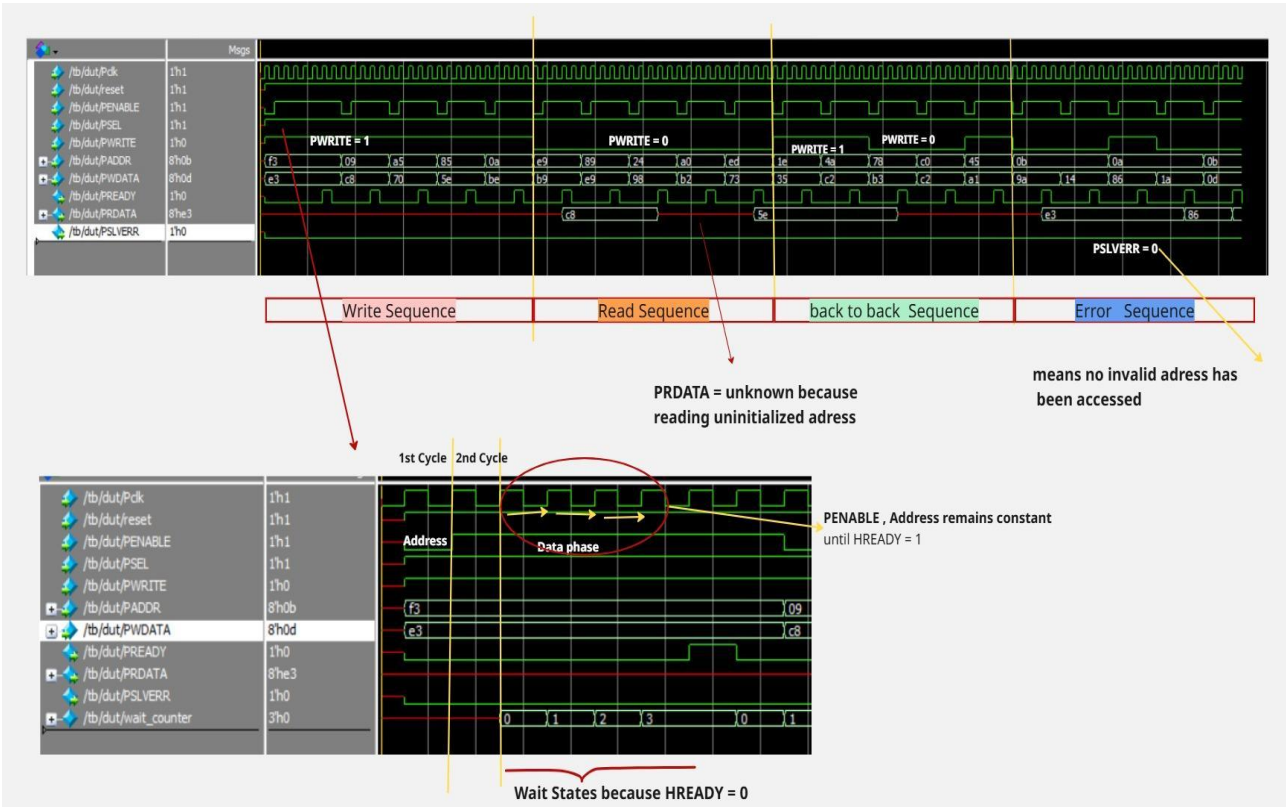


APB Results and Debugging- Prerna Agarwal

Simulation Result:

1st RUN



Functional Coverage Analysis

Metric	Covered	Total	% Hit	Status
Overall Covergroup	15	18	83.33%	✗ Uncovered

Coverpoint Cp_write			Coverpoint cp_data		
Bin	Hits	Status	Bin	Hits	Status
READ	21	✓ Covered	Zero	21	✓ Covered
WRITE	19	✓ Covered	others	19	✓ Covered
Coverpoints cp_addr			Coverpoint cp_error		
Bin	Hits	status	Bin	Hits	
Low (0-15)	13	✓ Covered	no_error	40	✓ Covered
Mid (16-127)	10	✓ Covered	error	0	✗ Missing
High(128-255)	17	✓ Covered			

Cross Coverage - cp_write x cp_addr

Combination	Hits	Status
READ, low	7	✓ Covered
WRITE, low	6	✓ Covered
READ, mid	4	✓ Covered
WRITE, mid	6	✓ Covered
READ, high	10	✓ Covered
WRITE, high	7	✓ Covered

Cross coverage = cp_write x cp_error

Combination	Hits	Status
READ, no_error	21	✓ Covered
WRITE, no_error	19	✓ Covered
READ, error	0	✗ Missing
WRITE, error	0	✗ Missing

Problems :

- 1) The current Environment has not generated PSLVERR = 1 means NO ERROR Transaction occurred.
- 2) Reading to uninitialized Address (showing Warnings/Error)
- 3) Error scenarios are not covered completely. hence ,Coverage results 83%

Approaches to Solve Problems:

- 1) Generate Error Scenarios from DUT or sequence

1. Modify Slave if(PADDR > max_valid_addr) PSLVERR = 1;	2. Force invalid access in sequence tr.PADDR = 'hFFFF; //invalid addr Or Add constraint Constraint invalid_addr{ PADDR inside {[256:300]}; }
--	---

So I added this constraint in transaction class , Now DUT(slave) must implement this Behaviour to Flag this invalid access. Because COVERAGE can be close to 100% if DUT actually produces that behavior.

2) Fixing - “reading to uninitialized address”

Intuition- disable address randomization while reading to only test addresses which are written.



- ✗ Weak Verification Quality
- ✗ Lose corner case testing
- ✗ Never hit Error(PSLVERR)
- ✗ Testing to written addresses only -leads to incomplete behaviour

Actually we need to decide how our DUT is supposed to behave then model and test it

A. Return 0/x (PSLVERR = 0) Current my DUT is behaving this	B. Signal error PSLVERR=1 meaning Data may be invalid if reading to uninitialized address occurs
--	--

Solution applied :

In DUT: should define specification

```
module apb_slave #(parameter int MAX_VALID_ADDR = 255)
    if(PWRITE)begin
        if( PADDR >= MAX_VALID_ADDR)begin
            PSLVERR <= 1; //master tried to access invalid address
        end else begin
            mem[PADDR[2:0]] <= PWDATA; //write
        end
    end
end
```

```

end
else begin
    if(PADDR >= MAX_VALID_ADDR)begin
        PSLVERR <=1;
    end else begin
        PRDATA <= mem[PADDR[2:0]]; //read
    end
end
end

```

In TB: should follow those specification

```

parameter int MAX_VALID_ADDR = 255;

// pass to UVM world
initial begin
    uvm_config_db#(int)::set(null, "*", "max_valid_addr", MAX_VALID_ADDR);
end

```

In Transaction:

```

int max_valid_addr;

function void pre_randomize();
    if (!uvm_config_db#(int)::get(null, "", "max_valid_addr", max_valid_addr))
        `uvm_fatal("CFG", "max_valid_addr not set");
endfunction

constraint invalid_addr {
    PADDR inside {[max_valid_addr : max_valid_addr + 155]};
}

```

2nd RUN , Simulation Results:

Functional Coverage:

Metric	Covered	Total	% Hit	Status
Overall Covergroup	17	18	97.22%	✖ Uncovered

Coverpoint Cp_write			Coverpoint cp_data		
Bin	Hits	Status	Bin	Hits	Status
READ	21	✓ Covered	Zero	21	✓ Covered
WRITE	19	✓ Covered	others	19	✓ Covered
Coverpoints cp_addr			Coverpoint cp_error		
Bin	Hits	status	Bin	Hits	
Low (0-15)	4	✓ Covered	no_error	26	✓ Covered
Mid (16-127)	14	✓ Covered	error	14	✓ Covered
High(128-255)	22	✓ Covered			

Combination	Hits	Status
READ, low	0	missing
WRITE, low	4	✓ Covered
READ, mid	8	✓ Covered
WRITE, mid	6	✓ Covered
READ, high	13	✓ Covered
WRITE, high	9	✓ Covered

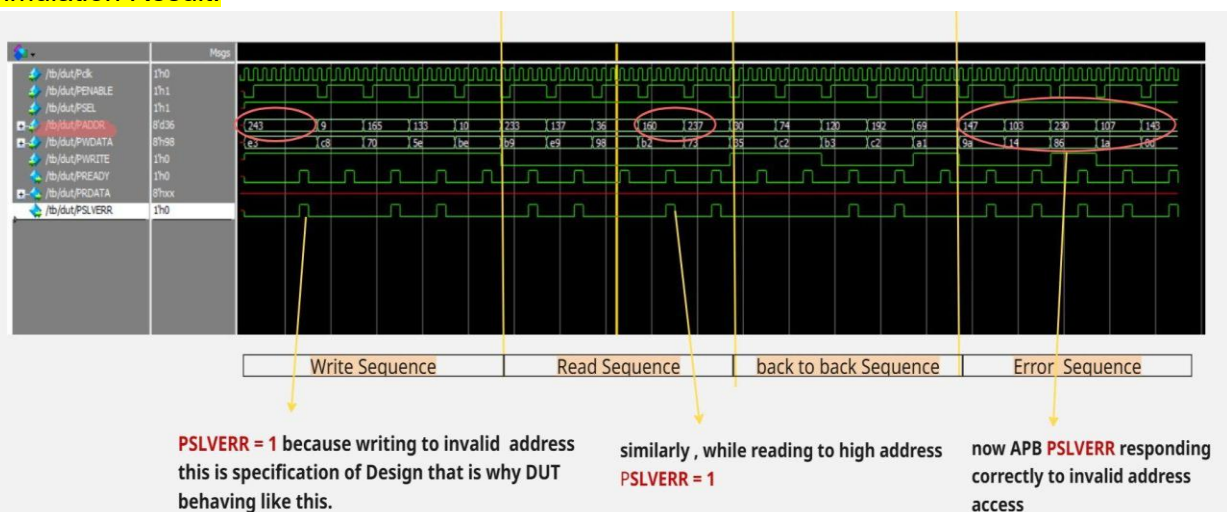
Combination	Hits	Status
READ, no_error	11	✓ Covered
WRITE, no_error	15	✓ Covered
READ, error	10	✓ Covered
WRITE, error	4	✓ Covered

Summary of coverage result:

No read Transaction generated in Low address range (READxlow address)

✓ Error handling **fully verified**

Simulation Result:



Adding Assertions:

```
//^ include "interface.sv"

module apb_assertions(apb_inf inf);

    //default clock
    default clocking cb @(posedge inf.Pclk);
    endclocking

    // 1) setup- address phase
    property setup_to_address;
        disable iff (!inf.reset)
        (inf.PSEL && !inf.PENABLE) |>=>(inf.PSEL && inf.PENABLE);
    endproperty

    ass_setup_to_address: assert property(setup_to_address);
    //else $error("APB: setup to address phase not followed");

    //PENABLE only high when slave is selected
    property penable_implies_psel;
        disable iff (!inf.reset)
        (inf.PENABLE) |-> (inf.PSEL);
    endproperty

    ass_penable_implies_psel: assert property (penable_implies_psel);

    // 2) during wait state - PENABLE constant
    property p_wait_state_hold;
        disable iff (!inf.reset)
        (inf.PSEL && inf.PENABLE && !inf.PREADY) |>=> (inf.PENABLE && inf.PSEL);
    endproperty

    ass_wait_state_hold: assert property(p_wait_state_hold);

    //PENABLE should drop after PREADY
    property p_complete_transfer;
        disable iff (!inf.reset)
        (inf.PSEL && inf.PENABLE && inf.PREADY) |>=> (!inf.PENABLE);
    endproperty

    ass_complete_transfer : assert property (p_complete_transfer);

    //3 signal stability -
    //Address Control signals stable during PENABLE is high

    property p_ctrl_stable;
        disable iff (!inf.reset)
        (inf.PSEL && inf.PENABLE && !inf.PREADY) |-> $stable({inf.PADDR , inf.PWRITE});
    endproperty
```



```

ass_ctrl_stable : assert property (p_ctrl_stable);

//PWDATA should be stable if PREADY = 0
property p_Wdata_stable;

    disable iff (!inf.reset)
    (inf.PSEL && inf.PENABLE && inf.PWRITE && !inf.PREADY) |-> $stable(inf.PWDATA);
endproperty
ass_Wdata_stable: assert property(p_Wdata_stable);

//Read data valid when PREADY high
property p_Rdata_valid;
    disable iff(!inf.reset)
    (inf.PSEL && inf.PENABLE && !inf.PWRITE && inf.PREADY) |->
!$isunknown(inf.PRDATA);
endproperty

ass_Rdata_valid: assert property(p_Rdata_valid);

//4) basic rules of APB

// when slave is not selected then PENABLE not asserted
property p_idle_rule;
    disable iff (!inf.reset)
    (!inf.PSEL) |-> (!inf.PENABLE);
endproperty
ass_idle_rule:assert property(p_idle_rule);

// when slave is selected then PADDR , PWRITE , PENABLE must not be X

property p_ctrl_notunknown;
    disable iff (!inf.reset)
    inf.PSEL |-> !$isunknown({inf.PADDR , inf.PWRITE, inf.PENABLE});
endproperty

ass_ctrl_notunknown: assert property(p_ctrl_notunknown);

// 5) ERROR handling

//no error during wait states
property p_noerror_wait;
    disable iff(!inf.reset)
    (inf.PSEL && inf.PENABLE && !inf.PREADY) |-> (inf.PSLVERR == 0);
endproperty

ass_noerror_wait: assert property(p_noerror_wait);

endmodule

In TB:

Bind apb_inf apb_assertions apb_assert_inst (inf);

```

Error encountered :

```
# Top level modules:
#      tb
# End time: 22:51:18 on May 04,2026, Elapsed time: 0:00:00
# Errors: 0, Warnings: 0
# vsim -voptargs="+acc" tb "+UVM_TESTNAME=apb_test"
# Start time: 22:50:35 on May 04,2026
# ** Note: (vsim-3812) Design is being optimized...
# ** Error: tb.sv(14): (vopt-2907) Parameterizing 'inf' has exceeded the recursion depth
limit of 10000.
# Optimization failed
```

Meaning:

The simulator got stuck in an **infinite recursive reference** involving **inf** (your interface)

I wrote: `module apb_assertions(apb_inf inf);`

Then bind: `bind apb_inf apb_assertions apb_assert_inst (inf);`

When we **bind apb_inf ...**, the tool tries to:

1. Attach **apb_assertions** to every instance of **apb_inf**
2. But your module itself takes **apb_inf inf** as a port
3. That creates **recursive elaboration of interface inside itself**

👉 Result:

interface → assertion → interface → assertion → ... (infinite)

Error resolving-> remove bind and instantiate assertion module in TB .

This avoids recursion because:

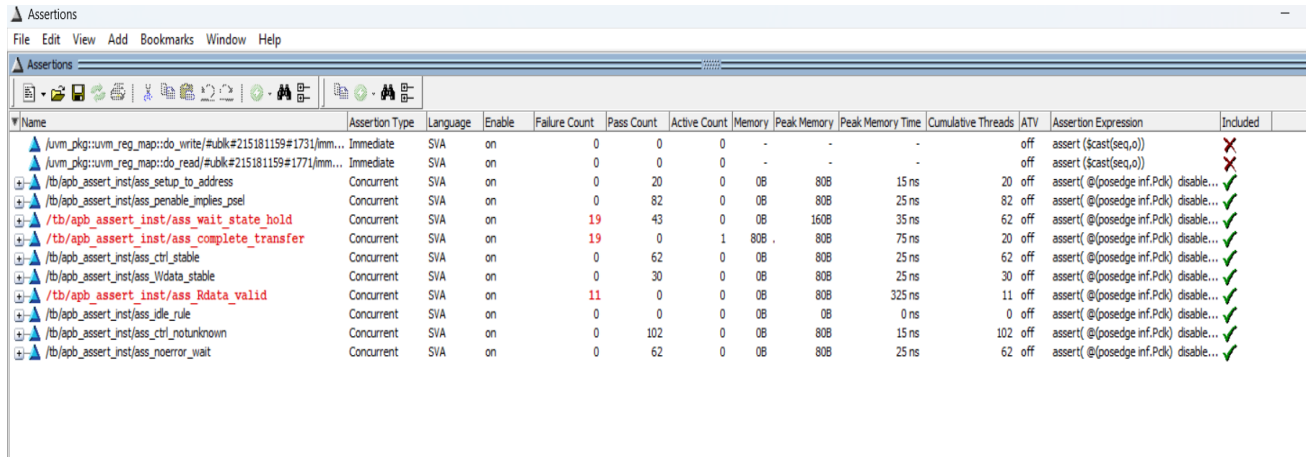
- You are not binding to interface type
- You are connecting **actual instance**

```
apb_assertions apb_assert_inst (inf);
```

Commands :

```
vsim -voptargs="+acc" -assertdebug tb  
run -all
```

View Assertion Report:



Name	Assertion Type	Language	Enable	Failure Count	Pass Count	Active Count	Memory	Peak Memory	Peak Memory Time	Cumulative Threads	ATV	Assertion Expression	Included
/uvm_pkg::uvm_reg_map::do_write/#ublk#215181159#1731/imm...	Immediate	SVA	on	0	0	0	-	-	-	-	off	assert (\$cast(seq,o))	✗
/uvm_pkg::uvm_reg_map::do_read/#ublk#215181159#1771/imm...	Immediate	SVA	on	0	0	0	-	-	-	-	off	assert (\$cast(seq,o))	✗
/tb/apb_assert_inst/ass_setup_to_address	Concurrent	SVA	on	0	20	0	0B	80B	15 ns	20	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_penable_implies_psel	Concurrent	SVA	on	0	82	0	0B	80B	25 ns	82	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_wait_state_hold	Concurrent	SVA	on	19	43	0	0B	160B	35 ns	62	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_complete_transfer	Concurrent	SVA	on	19	0	1	80B	80B	75 ns	20	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_ctrl_stable	Concurrent	SVA	on	0	62	0	0B	80B	25 ns	62	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_wdata_stable	Concurrent	SVA	on	0	30	0	0B	80B	25 ns	30	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_rdata_valid	Concurrent	SVA	on	11	0	0	0B	80B	325 ns	11	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_idle_rule	Concurrent	SVA	on	0	0	0	0B	0B	0 ns	0	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_ctrl_notunknown	Concurrent	SVA	on	0	102	0	0B	80B	15 ns	102	off	assert(@(posedge inf.Pck) disable...	✓
/tb/apb_assert_inst/ass_noerror_wait	Concurrent	SVA	on	0	62	0	0B	80B	25 ns	62	off	assert(@(posedge inf.Pck) disable...	✓

We can see assertion errors in transcript and debug it by capturing exact time given in transcript and Inspect the waveform.